

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий
Кафедра КБ-4 «Интеллектуальные системы информационной
безопасности»

Отчет по практической работе № 7
по дисциплине «Специальные разделы математики»
Тема: «Методы прогнозирования временных рядов»

Выполнил: студент группы ББМО-01-25

Мухаметшин Александр Ринатович

Проверила:

Изотова Анастасия Андреевна

Москва, 2025

Ход работы

Задание 2. Прогнозирование с оптимизацией параметров

Условие:

Дан временной ряд курса валюты за 15 дней:

[65.5, 66.2, 65.8, 67.1, 68.3, 67.9, 69.2, 70.5, 69.8, 71.2, 72.5, 71.8, 73.1, 74.3, 73.6]

Требуется:

1. Найти оптимальное значение параметра α для экспоненциального сглаживания (перебором от 0.1 до 0.9 с шагом 0.1), минимизирующее MSE
2. Построить прогноз на 3 дня вперед с оптимальным α
3. Построить график зависимости MSE от α
4. Сравнить качество прогноза с наивной моделью ($\alpha = 1$)

Решение:

```
currency = [65.5, 66.2, 65.8, 67.1, 68.3, 67.9, 69.2, 70.5, 69.8,
71.2, 72.5, 71.8, 73.1, 74.3, 73.6]
print(f"\nВременной ряд курса валюты: {currency}")

def exponential_smoothing(data, alpha):
    n = len(data)
    smoothed = [0.0] * n
    forecasts = [0.0] * n
    errors = [0.0] * n

    smoothed[0] = data[0]
    forecasts[0] = data[0]

    for t in range(1, n):
        forecasts[t] = smoothed[t-1]
        errors[t] = data[t] - forecasts[t]
        smoothed[t] = alpha * data[t] + (1 - alpha) * smoothed[t-1]

    return smoothed, forecasts, errors

def calculate_mse(errors):
    return np.mean(np.array(errors[1:])**2)

# 1. Поиск оптимального  $\alpha$ 
print("\n1. Поиск оптимального  $\alpha$ :")
alphas = np.arange(0.1, 1.0, 0.1)
mse_values = []

for alpha in alphas:
```

```

_, _, errors = exponential_smoothing(currency, alpha)
mse = calculate_mse(errors)
mse_values.append(mse)
print(f"α = {alpha:.1f}, MSE = {mse:.4f}")

optimal_alpha = alphas[np.argmin(mse_values)]
optimal_mse = min(mse_values)
print(f"\nОптимальное α = {optimal_alpha:.1f} с MSE = {optimal_mse:.4f}")

# 2. Прогноз на 3 дня вперед (многошаговый)
smoothed_opt, _, _ = exponential_smoothing(currency, optimal_alpha)

print(f"\n2. Прогноз на 3 дня вперед с α = {optimal_alpha:.1f}:")
forecast_16 = smoothed_opt[-1]
forecast_17 = optimal_alpha * forecast_16 + (1 - optimal_alpha) * smoothed_opt[-1]
forecast_18 = optimal_alpha * forecast_17 + (1 - optimal_alpha) * forecast_17

print(f"День 16: {forecast_16:.2f}")
print(f"День 17: {forecast_17:.2f}")
print(f"День 18: {forecast_18:.2f}")

# 3. График MSE от α
plt.figure(figsize=(5, 3))
plt.plot(alphas, mse_values, 'o-', linewidth=2, markersize=8)
plt.axvline(x=optimal_alpha, color='r', linestyle='--', label=f'Оптимальное α = {optimal_alpha:.1f}')
plt.xlabel('Параметр сглаживания α')
plt.ylabel('MSE')
plt.title('Зависимость MSE от параметра α')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

# 4. Сравнение с наивной моделью (α = 1)
_, forecasts_naive, errors_naive = exponential_smoothing(currency, 1.0)
mse_naive = calculate_mse(errors_naive)

print(f"\n4. Сравнение с наивной моделью (α = 1.0):")
print(f"Оптимальная модель (α = {optimal_alpha:.1f}): MSE = {optimal_mse:.4f}")

```

```
print(f"Наивная модель ( $\alpha = 1.0$ ): MSE = {mse_naive:.4f}")
print(f"Улучшение: {(mse_naive - optimal_mse) / mse_naive * 100} :.2f}%")
```

Вывод:

Временной ряд курса валюты: [65.5, 66.2, 65.8, 67.1, 68.3, 67.9, 69.2, 70.5, 69.8, 71.2, 72.5, 71.8, 73.1, 74.3, 73.6]

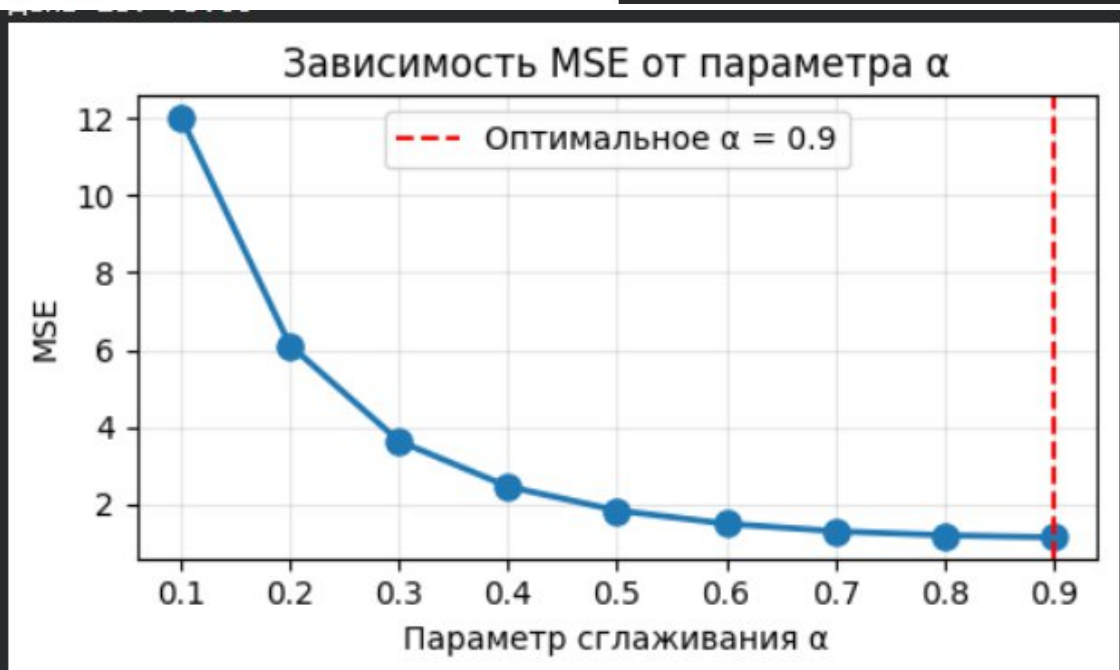
1. Поиск оптимального α :

$\alpha = 0.1$, MSE = 12.0432
 $\alpha = 0.2$, MSE = 6.1176
 $\alpha = 0.3$, MSE = 3.6279
 $\alpha = 0.4$, MSE = 2.4438
 $\alpha = 0.5$, MSE = 1.8248
 $\alpha = 0.6$, MSE = 1.4815
 $\alpha = 0.7$, MSE = 1.2872
 $\alpha = 0.8$, MSE = 1.1806
 $\alpha = 0.9$, MSE = 1.1288

Оптимальное $\alpha = 0.9$ с MSE = 1.1288

2. Прогноз на 3 дня вперед с $\alpha = 0.9$:

День 16: 73.66
 День 17: 73.66
 День 18: 73.66



4. Сравнение с наивной моделью ($\alpha = 1.0$):

Оптимальная модель ($\alpha = 0.9$): MSE = 1.1288

Наивная модель ($\alpha = 1.0$): MSE = 1.1121

Улучшение: -1.50%

Задание 3. Модель ARMA(1,1)

Условие:

Дан временной ряд:

[100, 105, 108, 112, 115, 119, 122, 126, 129, 133, 136, 140]

Требуется:

1. Построить модель ARMA(1,1):

$$s(t) = a_1 \cdot s(t-1) + \varepsilon(t) + b_1 \cdot \varepsilon(t-1)$$

2. Оценить параметры a_1 и b_1 методом наименьших квадратов
3. Вычислить остатки $\varepsilon(t)$
4. Проверить адекватность модели (проверка остатков на некоррелированность)
5. Построить прогноз на 4 шага вперед

Решение:

```
arma_data = [100, 105, 108, 112, 115, 119, 122, 126, 129, 133, 136, 140]
print(f"\nВременной ряд: {arma_data}")

# Упрощенная оценка для ARMA(1,1)
# Сначала оцениваем AR(1) часть
X = np.array(arma_data[:-1])
y = np.array(arma_data[1:])
X_with_const = np.column_stack([np.ones(len(X)), X])
coeffs = np.linalg.lstsq(X_with_const, y, rcond=None)[0]
a0, a1 = coeffs

print("\n1. Оценка параметров (упрощенный метод):")
print(f"a1 = {a1:.4f}")
print(f"a0 = {a0:.4f}")

# Вычисление остатков
residuals = [0]
for t in range(1, len(arma_data)):
    pred = a0 + a1 * arma_data[t-1]
    residuals.append(arma_data[t] - pred)

print(f"\n2. Остатки  $\varepsilon(t)$ : {[f'{r:.2f}' for r in residuals[1:]]}")

# Проверка автокорреляции остатков
residuals_array = np.array(residuals[1:])
mean_resid = np.mean(residuals_array)
autocorr = np.correlate(residuals_array - mean_resid, residuals_array - mean_resid, mode='full')
autocorr = autocorr[len(autocorr)//2:] / autocorr[len(autocorr)//2]

print(f"\n3. Автокорреляция остатков (lag=1): {autocorr[1]:.4f}")
if abs(autocorr[1]) < 0.3:
    print("Остатки слабо коррелированы - модель адекватна")
```

```

else:
    print("Остатки коррелированы - возможно нужна более сложная модель")

# 4. Прогноз на 4 шага вперед
print("\n4. Прогноз на 4 шага вперед:")
last_value = arma_data[-1]
last_error = residuals[-1]

for k in range(1, 5):
    forecast = a0 + a1 * last_value
    print(f"Шаг {k}: {forecast:.2f}")
    last_value = forecast

```

Вывод:

```

Временной ряд: [100, 105, 108, 112, 115, 119, 122, 126, 129, 133, 136, 140]

1. Оценка параметров (упрощенный метод):
a1 = 0.9852
a0 = 5.3890

2. Остатки  $\epsilon(t)$ : ['1.09', '-0.84', '0.21', '-0.73', '0.31', '-0.63', '0.41', '-0.53', '0.52', '-0.42', '0.62']

3. Автокорреляция остатков (lag=1): -0.6819
Остатки коррелированы - возможно нужна более сложная модель

4. Прогноз на 4 шага вперед:
Шаг 1: 143.32
Шаг 2: 146.59
Шаг 3: 149.82
Шаг 4: 152.99

```

Задание 4. Адаптивная фильтрация с переменным окном

Условие:

Дан временной ряд с изменяющейся динамикой:

[50, 52, 54, 56, 58, 60, 75, 90, 92, 94, 96, 98, 100, 102, 104]

Требуется:

1. Реализовать адаптивную фильтрацию с автоматическим выбором размера окна λ
2. Критерий выбора: минимум скользящей MSE на окне из 5 последних наблюдений
3. Протестировать окна размером $\lambda = 2, 3, 4, 5$
4. Построить график изменения оптимального размера окна во времени
5. Сравнить с фиксированным окном $\lambda = 3$

Решение:

```

adaptive_data = [50, 52, 54, 56, 58, 60, 75, 90, 92, 94, 96, 98, 100, 102, 104]
print(f"\nВременной ряд: {adaptive_data}")

```

```

def adaptive_filter(data, window_size):
    n = len(data)
    weights = np.ones(window_size) / window_size
    forecasts = [0.0] * n
    errors = [0.0] * n

    for t in range(window_size, n):
        window = np.array(data[t-window_size:t])
        forecast = np.dot(weights, window)
        forecasts[t] = forecast
        error = data[t] - forecast
        errors[t] = error

        # Адаптация весов
        sum_squares = np.sum(window ** 2)
        if sum_squares > 0:
            k_g = 1.0 / sum_squares
            weights = weights + k_g * error * window
            # Нормализация
            weights = weights / np.sum(weights)

    return forecasts, errors

# 1-3. Тестирование разных размеров окна
print("\n1-3. Тестирование разных размеров окна:")
print(f"{'λ':>4} | {'MSE':>12} | {'MAE':>12}")

window_sizes = [2, 3, 4, 5]
results = {}

for ws in window_sizes:
    forecasts, errors = adaptive_filter(adaptive_data, ws)
    valid_errors = np.array([e for e in errors[ws:] if e != 0])
    mse = np.mean(valid_errors ** 2)
    mae = np.mean(np.abs(valid_errors))
    results[ws] = {'mse': mse, 'mae': mae, 'forecasts': forecasts,
                  'errors': errors}
    print(f"{ws:4d} | {mse:12.4f} | {mae:12.4f}")

best_window = min(results.keys(), key=lambda k:
                  results[k]['mse'])
print(f"\nОптимальный размер окна: λ = {best_window}")

# 4. График изменения оптимального размера окна
print("\n4. Визуализация результатов...")

```

```

# 5. Сравнение с фиксированным окном  $\lambda=3$ 
fixed_forecasts, fixed_errors = adaptive_filter(adaptive_data,
3)
fixed_mse = calculate_mse(fixed_errors)

print(f"\n5. Сравнение с фиксированным окном  $\lambda = 3$ :")
print(f"Лучший результат ( $\lambda = \{best\_window\}$ ): MSE = {results[best_window]['mse']:.4f}")
print(f"Фиксированное окно ( $\lambda = 3$ ): MSE = {fixed_mse:.4f}")

# График
fig, axes = plt.subplots(2, 2, figsize=(10, 6))

for idx, ws in enumerate(window_sizes):
    row = idx // 2
    col = idx % 2
    f = results[ws]['forecasts']
    axes[row, col].plot(adaptive_data, 'o-', label='Факт',
linewidth=2)
    axes[row, col].plot(range(ws, len(adaptive_data)), f[ws:],
's--', label='Прогноз', linewidth=2)
    axes[row, col].set_title(f' $\lambda = \{ws\}$ , MSE = {results[ws]["mse"]:.2f}')
    axes[row, col].legend()
    axes[row, col].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Вывод:

Временной ряд: [50, 52, 54, 56, 58, 60, 75, 90, 92, 94, 96, 98, 100, 102, 104]

1-3. Тестирование разных размеров окна:

λ	MSE	MAE
2	71.8715	5.9602
3	117.0164	8.2144
4	173.9160	10.6620
5	245.5514	13.3644

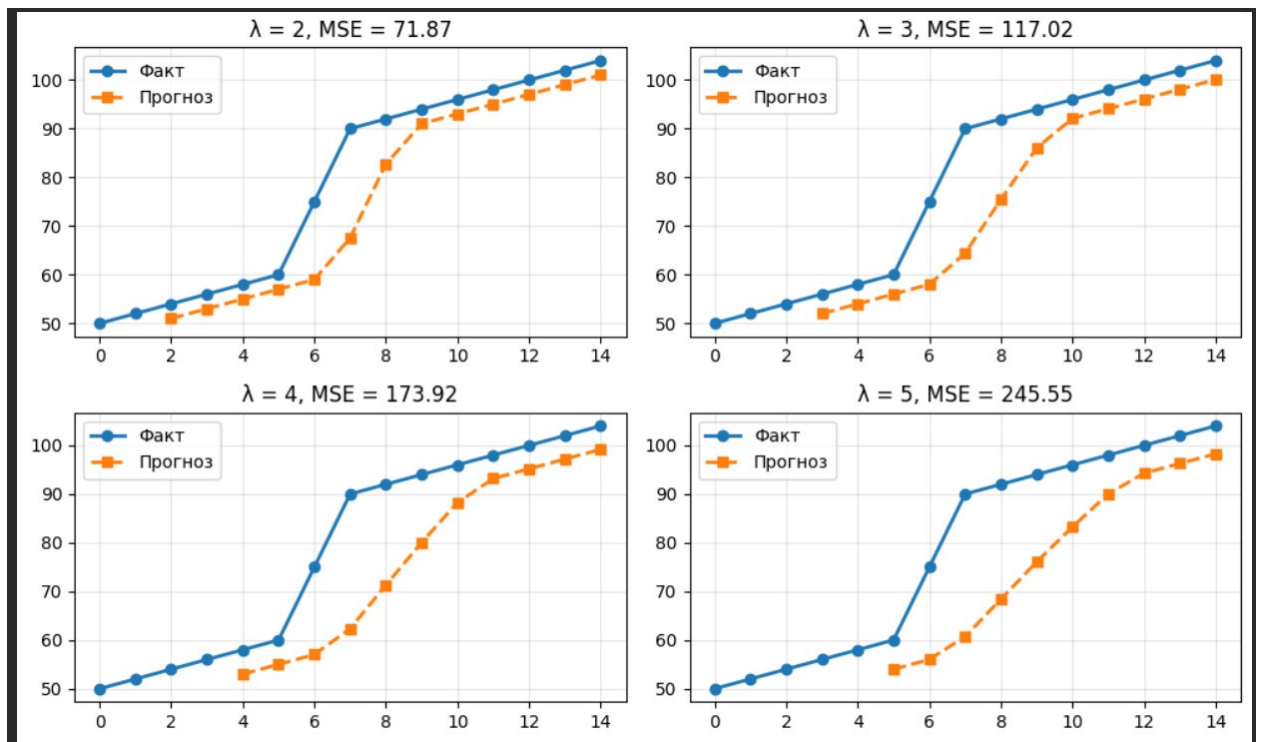
Оптимальный размер окна: $\lambda = 2$

4. Визуализация результатов...

5. Сравнение с фиксированным окном $\lambda = 3$:

Лучший результат ($\lambda = 2$): MSE = 71.8715

Фиксированное окно ($\lambda = 3$): MSE = 100.2998



Задание 5. Прогнозирование с учетом тренда

Условие:

Дан временной ряд с явным линейным трендом:

[10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65]

Требуется:

1. Реализовать метод двойного экспоненциального сглаживания (метод Хольта):

$$L(t) = \alpha \cdot s(t) + (1-\alpha) \cdot [L(t-1) + T(t-1)]$$

$$T(t) = \beta \cdot [L(t) - L(t-1)] + (1-\beta) \cdot T(t-1)$$

$$\hat{s}(t+k) = L(t) + k \cdot T(t)$$

где $L(t)$ — уровень, $T(t)$ — тренд

2. Использовать $\alpha = 0.3$, $\beta = 0.1$

3. Построить прогноз на 5 шагов вперед

4. Сравнить с простым экспоненциальным сглаживанием

5. Визуализировать компоненты уровня и тренда

Решение:

```
trend_data = [10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65]
print(f"\nВременной ряд: {trend_data}")

def holt_method(data, alpha=0.3, beta=0.1):
    n = len(data)
    level = [0.0] * n
    trend = [0.0] * n
    forecasts = [0.0] * n
```

```

errors = [0.0] * n

# Инициализация
level[0] = data[0]
trend[0] = data[1] - data[0]
forecasts[0] = data[0]

for t in range(1, n):
    forecasts[t] = level[t-1] + trend[t-1]
    errors[t] = data[t] - forecasts[t]
    level[t] = alpha * data[t] + (1 - alpha) * (level[t-1]
+ trend[t-1])
    trend[t] = beta * (level[t] - level[t-1]) + (1 - beta)
* trend[t-1]

return level, trend, forecasts, errors

# 1-2. Применение метода Хольта
alpha, beta = 0.3, 0.1
level, trend, forecasts_holt, errors_holt =
holt_method(trend_data, alpha, beta)
mse_holt = calculate_mse(errors_holt)

print(f"\n1-2. Метод Хольта с  $\alpha = \{alpha\}$ ,  $\beta = \{beta\}$ ")
print(f"MSE = {mse_holt:.4f}")
print(f"Текущий уровень L({len(trend_data)-1}) = {level[-1]:.2f}")
print(f"Текущий тренд T({len(trend_data)-1}) = {trend[-1]:.2f}")

# 3. Прогноз на 5 шагов вперед
print("\n3. Прогноз на 5 шагов вперед:")
for k in range(1, 6):
    forecast = level[-1] + k * trend[-1]
    print(f"Шаг {k}: {forecast:.2f}")

# 4. Сравнение с простым экспоненциальным сглаживанием
smoothed_simple, forecasts_simple, errors_simple =
exponential_smoothing(trend_data, alpha)
mse_simple = calculate_mse(errors_simple)

print(f"\n4. Сравнение с простым экспоненциальным
сглаживанием:")
print(f"Метод Хольта: MSE = {mse_holt:.4f}")
print(f"Простое ЭС: MSE = {mse_simple:.4f}")
print(f"Метод Хольта лучше на {((mse_simple - mse_holt) /
mse_simple * 100):.2f}%")

```

```

# 5. Визуализация компонент
fig, axes = plt.subplots(3, 1, figsize=(6, 10))

# График 1: Данные и прогнозы
axes[0].plot(trend_data, 'o-', label='Факт', linewidth=2,
markersize=8)
axes[0].plot(forecasts_holt, 's--', label='Метод Хольта',
linewidth=2)
axes[0].plot(forecasts_simple, '^:', label='Простое ЭС',
linewidth=2)
axes[0].set_title('Прогнозирование с трендом')
axes[0].set_ylabel('Значение')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# График 2: Уровень
axes[1].plot(level, 'g-', linewidth=2, marker='o')
axes[1].set_title('Компонента уровня L(t)')
axes[1].set_ylabel('Уровень')
axes[1].grid(True, alpha=0.3)

# График 3: Тренд
axes[2].plot(trend, 'r-', linewidth=2, marker='s')
axes[2].set_title('Компонента тренда T(t)')
axes[2].set_ylabel('Тренд')
axes[2].set_xlabel('Время')
axes[2].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Вывод:

```

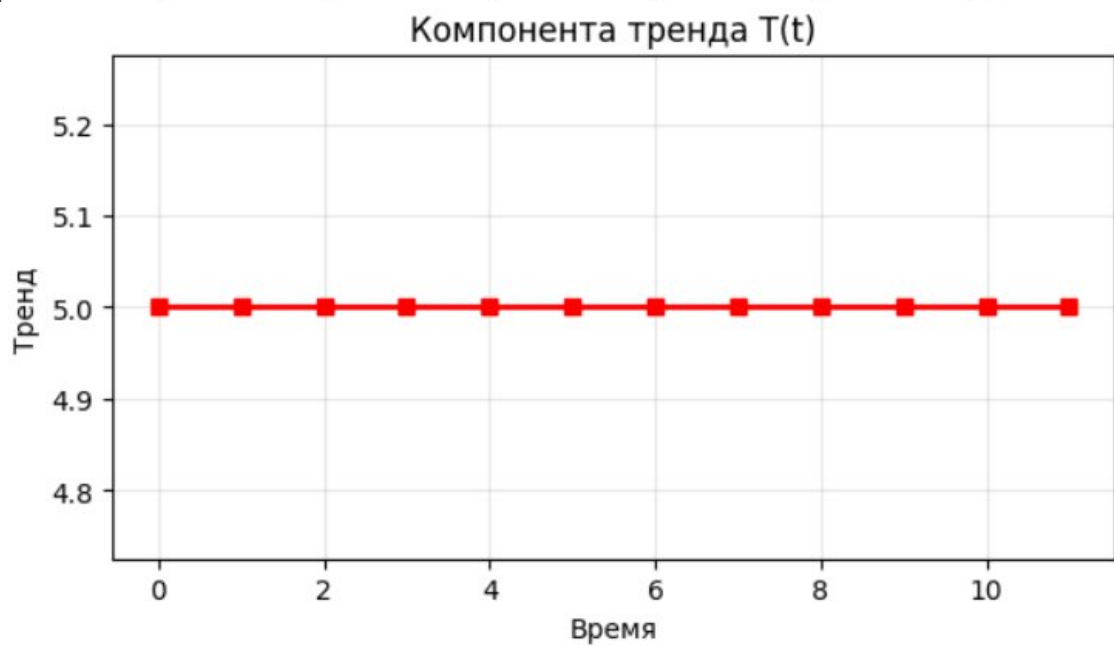
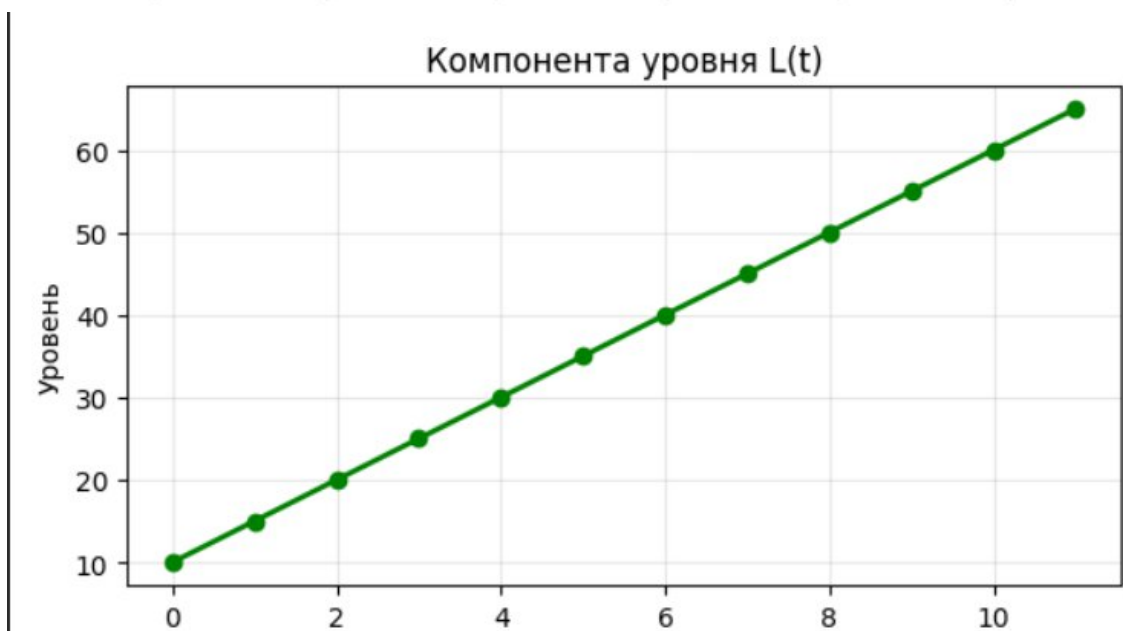
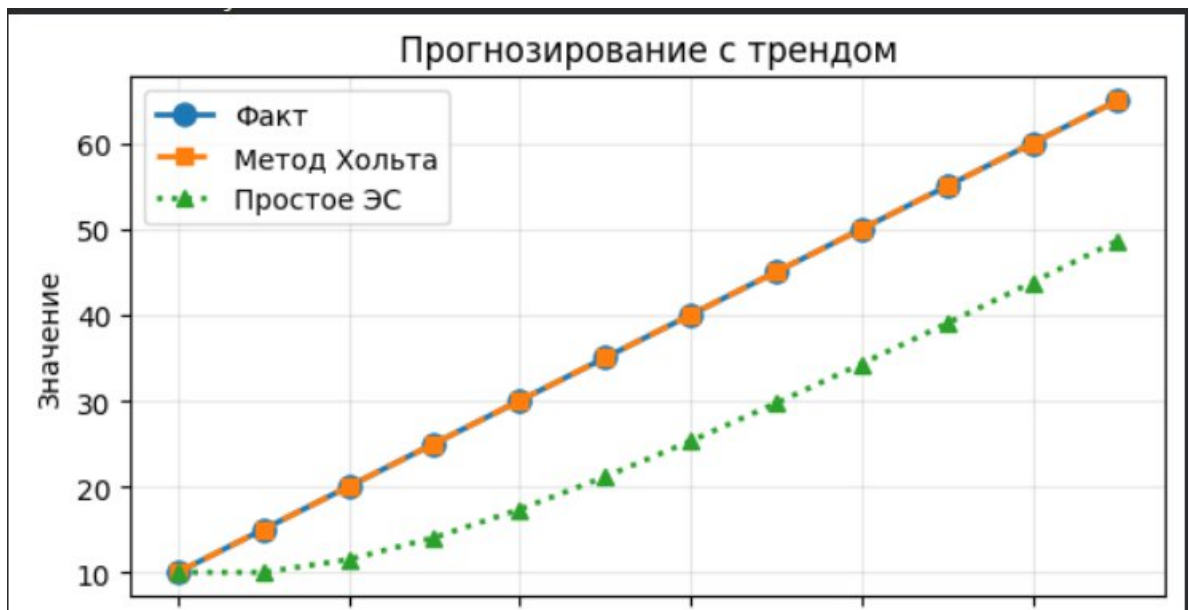
Временной ряд: [10, 15, 20, 25, 30, 35, 40, 45, 50, 55, 60, 65]

1-2. Метод Хольта с  $\alpha = 0.3$ ,  $\beta = 0.1$ 
MSE = 0.0000
Текущий уровень  $L(11) = 65.00$ 
Текущий тренд  $T(11) = 5.00$ 

3. Прогноз на 5 шагов вперед:
Шаг 1: 70.00
Шаг 2: 75.00
Шаг 3: 80.00
Шаг 4: 85.00
Шаг 5: 90.00

4. Сравнение с простым экспоненциальным сглаживанием:
Метод Хольта: MSE = 0.0000
Простое ЭС: MSE = 186.5156
Метод Хольта лучше на 100.00%

```



Задание 6. Комбинированное прогнозирование

Условие:

Дан временной ряд продаж:

[200, 210, 205, 220, 230, 225, 240, 250, 245, 260, 270, 265, 280, 290, 285]

Требуется:

1. Построить три модели:

- Экспоненциальное сглаживание ($\alpha = 0.3$)
- Адаптивная фильтрация ($\lambda = 3$)
- Линейная регрессия

2. Для каждой модели вычислить MSE

3. Построить комбинированный прогноз как взвешенное среднее:

$$\hat{s}_{\text{combined}} = \sum w_i \cdot \hat{s}_i$$

где веса $w_i = (1/\text{MSE}_i) / \sum (1/\text{MSE}_j)$

4. Сравнить качество комбинированного прогноза с отдельными моделями

5. Построить прогноз на 3 шага вперед

Решение:

```
sales_data = [200, 210, 205, 220, 230, 225, 240, 250, 245, 260, 270, 265, 280, 290, 285]
print(f"\nВременной ряд продаж: {sales_data}")

def linear_regression_forecast(data):
    n = len(data)
    X = np.arange(n)
    y = np.array(data)
    X_with_const = np.column_stack([np.ones(n), X])
    coeffs = np.linalg.lstsq(X_with_const, y, rcond=None)[0]
    a, b = coeffs
    forecasts = [a + b * i for i in range(n)]
    errors = [data[i] - forecasts[i] for i in range(n)]
    return forecasts, errors, a, b

# 1. Три модели
print("\n1. Построение трех моделей:")

# Модель 1: Экспоненциальное сглаживание
_, forecasts_es, errors_es = exponential_smoothing(sales_data, 0.3)
mse_es = calculate_mse(errors_es)
print(f"Экспоненциальное сглаживание: MSE = {mse_es:.4f}")

# Модель 2: Адаптивная фильтрация
forecasts_af, errors_af = adaptive_filter(sales_data, 3)
```

```

mse_af = calculate_mse(errors_af)
print(f"Адаптивная фильтрация: MSE = {mse_af:.4f}")

# Модель 3: Линейная регрессия
forecasts_lr, errors_lr, a, b =
linear_regression_forecast(sales_data)
mse_lr = calculate_mse(errors_lr)
print(f"Линейная регрессия: MSE = {mse_lr:.4f}")

# 2-3. Комбинированный прогноз
print("\n2-3. Комбинированный прогноз:")

# Веса пропорциональны обратным MSE
w_es = 1 / mse_es
w_af = 1 / mse_af
w_lr = 1 / mse_lr
total = w_es + w_af + w_lr

w_es /= total
w_af /= total
w_lr /= total

print(f"Веса моделей:")
print(f" ЭС: {w_es:.4f}")
print(f" АФ: {w_af:.4f}")
print(f" ЛР: {w_lr:.4f}")

# Комбинированные прогнозы
forecasts_combined = []
for i in range(len(sales_data)):
    combined = w_es * forecasts_es[i] + w_af * forecasts_af[i]
    + w_lr * forecasts_lr[i]
    forecasts_combined.append(combined)

errors_combined = [sales_data[i] - forecasts_combined[i] for i
in range(len(sales_data))]
mse_combined = calculate_mse(errors_combined)

# 4. Сравнение
print(f"\n4. Сравнение качества:")
print(f"Комбинированная модель: MSE = {mse_combined:.4f}")
improvement_es = ((mse_es - mse_combined) / mse_es * 100)
improvement_af = ((mse_af - mse_combined) / mse_af * 100)
improvement_lr = ((mse_lr - mse_combined) / mse_lr * 100)
print(f"Улучшение относительно ЭС: {improvement_es:.2f}%")
print(f"Улучшение относительно АФ: {improvement_af:.2f}%")

```

```

print(f"Улучшение относительно ЛР: {improvement_lr:.2f}%")

# 5. Прогноз на 3 шага вперед
print("\n5. Прогноз на 3 шага вперед:")
n = len(sales_data)

# Для ЭС
last_smoothed = forecasts_es[-1]
future_es = [last_smoothed] * 3

# Для АФ
future_af = []
extended_data = sales_data.copy()
for k in range(3):
    window = extended_data[-3:]
    weights = np.ones(3) / 3
    forecast = np.dot(weights, window)
    future_af.append(forecast)
    extended_data.append(forecast)

# Для ЛР
future_lr = [a + b * (n + k) for k in range(1, 4)]

# Комбинированный
for k in range(3):
    combined = w_es * future_es[k] + w_af * future_af[k] + w_lr
    * future_lr[k]
    print(f"Шаг {k+1}: {combined:.2f}")

# График
plt.figure(figsize=(8, 6))
plt.plot(sales_data, 'o-', label='Факт', linewidth=2,
markersize=8)
plt.plot(forecasts_es, 's--', label='ЭС', linewidth=1,
alpha=0.7)
plt.plot(forecasts_af, '^--', label='АФ', linewidth=1,
alpha=0.7)
plt.plot(forecasts_lr, 'v--', label='ЛР', linewidth=1,
alpha=0.7)
plt.plot(forecasts_combined, 'd-', label='Комбинированная',
linewidth=2, markersize=6)
plt.xlabel('Время')
plt.ylabel('Продажи')
plt.title('Комбинированное прогнозирование')
plt.legend()
plt.grid(True, alpha=0.3)

```

```
plt.tight_layout()
plt.show()
```

Вывод:

Временной ряд продаж: [200, 210, 205, 220, 230, 225, 240, 250, 245, 260, 270, 265, 280, 290, 285]

1. Построение трех моделей:

Экспоненциальное сглаживание: $MSE = 375.3027$

Адаптивная фильтрация: $MSE = 172.5803$

Линейная регрессия: $MSE = 25.3229$

2-3. Комбинированный прогноз:

Веса моделей:

ЭС: 0.0556

АФ: 0.1208

ЛР: 0.8236

4. Сравнение качества:

Комбинированная модель: $MSE = 114.2761$

Улучшение относительно ЭС: 69.55%

Улучшение относительно АФ: 33.78%

Улучшение относительно ЛР: -351.28%

5. Прогноз на 3 шага вперед:

Шаг 1: 299.59

Шаг 2: 305.16

Шаг 3: 310.39

